

# BAB 2

## KONSEP DASAR PEMROGRAMAN BERORIENTASI OBJEK

### 2.1 Pendahuluan

Pada BAB 1, mahasiswa telah mempelajari dasar-dasar Java, seperti tipe data, variabel, operator, struktur kondisional, perulangan, dan array. Dasar tersebut sangat penting karena menjadi bekal awal sebelum masuk ke inti mata kuliah Pemrograman Berorientasi Objek.

Pada BAB 2 ini, pembahasan mulai masuk ke cara berpikir **berorientasi objek**. Jika pada pemrograman dasar kita lebih banyak fokus pada langkah-langkah program, maka pada pemrograman berorientasi objek kita mulai melihat program sebagai kumpulan objek yang saling berinteraksi untuk menyelesaikan suatu masalah.

Dengan memahami konsep ini, mahasiswa tidak hanya belajar menulis kode yang berjalan, tetapi juga belajar membangun program yang lebih rapi, terstruktur, dan mudah dikembangkan. Karena itu, BAB 2 menjadi jembatan penting antara dasar Java dan materi inti PBO pada bab-bab berikutnya.

### 2.2 Tujuan Pembelajaran

Setelah menyelesaikan praktikum pada bab ini, mahasiswa diharapkan mampu memahami konsep dasar pemrograman berorientasi objek dan mengimplementasikannya dalam program Java sederhana.

Secara lebih rinci, mahasiswa diharapkan mampu:

1. Menjelaskan pengertian pemrograman berorientasi objek.
2. Menjelaskan karakteristik dasar OOP.
3. Memahami konsep class dan object.
4. Memahami atribut, method, field, dan constructor.
5. Menggunakan modifier sederhana pada class, atribut, dan method.
6. Menggunakan keyword `this` pada program Java.
7. Membuat program berbasis objek dengan contoh yang dekat dengan lingkungan ITERA.

## 2.3 Pengenalan Pemrograman Berorientasi Objek

Pemrograman berorientasi objek atau **Object Oriented Programming (OOP)** adalah pendekatan pemrograman yang memodelkan program sebagai kumpulan objek. Setiap objek memiliki data dan perilaku, lalu objek-objek tersebut dapat saling berinteraksi untuk menjalankan fungsi program.

Dalam pendekatan ini, kita tidak hanya memikirkan “langkah apa yang harus dikerjakan program”, tetapi juga memikirkan “objek apa saja yang ada di dalam sistem”. Misalnya, jika kita ingin membuat sistem akademik sederhana di ITERA, maka kita bisa memikirkan objek seperti **Mahasiswa**, **Dosen**, **MataKuliah**, dan **Kelas**.

Pendekatan ini membuat program lebih mudah dipahami karena strukturnya lebih dekat dengan dunia nyata. Kita terbiasa mengenal entitas seperti mahasiswa, dosen, ruang kelas, atau laboratorium, sehingga konsep class dan object biasanya terasa lebih masuk akal dibanding hanya melihat program sebagai kumpulan instruksi.

## 2.4 Karakteristik Dasar OOP

Pemrograman berorientasi objek memiliki beberapa karakteristik utama. Pada tahap awal, mahasiswa belum perlu menghafal semuanya secara terlalu teknis, tetapi perlu memahami gambaran besarnya terlebih dahulu.

### 2.4.1 Abstraksi

Abstraksi adalah proses mengambil hal-hal yang penting dari suatu objek dan mengabaikan detail yang belum diperlukan. Dengan abstraksi, kita fokus pada ciri dan perilaku utama dari suatu objek.

Sebagai contoh, jika kita membuat class `Mahasiswa`, maka kita cukup mengambil atribut penting seperti `nama`, `nim`, dan `prodi`. Kita tidak perlu langsung memasukkan semua hal tentang mahasiswa, misalnya nomor sepatu, warna tas, atau tempat duduk favorit di kelas, karena itu tidak relevan untuk sistem yang sedang dibuat.

### 2.4.2 Enkapsulasi

Enkapsulasi adalah konsep membungkus data dan method ke dalam satu kesatuan, yaitu class. Konsep ini juga berkaitan dengan pembatasan akses terhadap data agar tidak bisa diubah sembarangan dari luar.

Sebagai contoh, data nilai mahasiswa sebaiknya tidak diubah langsung oleh sembarang bagian program. Data tersebut lebih aman jika diakses melalui method tertentu, seperti `setNilai()` atau `getNilai()`, sehingga class dapat mengatur bagaimana data digunakan.

### **2.4.3 Pewarisan (*Inheritance*)**

Pewarisan adalah konsep di mana sebuah class dapat mewarisi atribut dan method dari class lain. Dengan cara ini, kita tidak perlu menulis ulang kode yang sebenarnya sudah dimiliki oleh class induk.

Misalnya, kita bisa memiliki class `CivitasAkademika` yang berisi atribut umum seperti nama dan id. Lalu dari class tersebut dapat diturunkan class `Mahasiswa` dan `Dosen`, karena keduanya sama-sama bagian dari civitas akademika.

### **2.4.4 Polymorphism**

Polymorphism berarti satu bentuk dapat memiliki banyak perilaku. Dalam Java, konsep ini sering terlihat pada method dengan nama yang sama tetapi memiliki perilaku berbeda sesuai konteks.

Sebagai contoh sederhana, method `tampilkanInfo()` pada class `Mahasiswa` dan `Dosen` bisa memiliki isi yang berbeda. Nama method-nya sama, tetapi hasil yang ditampilkan bisa menyesuaikan jenis objeknya.

### **2.4.5 Reusability**

Reusability berarti kode yang sudah dibuat dapat digunakan kembali. Ini adalah salah satu keuntungan besar dari OOP, karena programmer tidak perlu selalu memulai dari nol ketika membuat program baru.

Misalnya, class `Mahasiswa` yang dibuat untuk latihan praktikum bisa dikembangkan lagi untuk tugas besar, mini project, atau sistem akademik yang lebih lengkap. Dengan demikian, kode yang sudah ditulis tidak terbuang percuma.

### **2.4.6 Message atau Komunikasi Antarobjek**

Dalam OOP, objek tidak bekerja sendirian. Objek saling berinteraksi melalui pemanggilan method, dan interaksi ini dapat dipahami sebagai pengiriman pesan dari satu objek ke objek lain.

Contohnya, objek `Mahasiswa` dapat memanggil method pada objek `MataKuliah` untuk melihat nama mata kuliah yang diambil. Jadi, program bukan hanya kumpulan data, tetapi juga kumpulan objek yang saling bekerja sama.

## 2.5 Class

Class adalah cetak biru atau rancangan untuk membuat objek. Di dalam class, kita mendefinisikan data apa saja yang dimiliki objek dan perilaku apa saja yang bisa dilakukan objek tersebut.

Jika dianalogikan dengan dunia nyata, class seperti rancangan umum, sedangkan object adalah bentuk nyatanya. Misalnya, class Mahasiswa adalah rancangan umum, sedangkan andi, budi, dan siti adalah object yang dibuat dari class tersebut.

Bentuk umum penulisan class di Java adalah sebagai berikut:

```
class NamaClass {  
  
    // atribut  
  
    // method  
  
}
```

Contoh class sederhana:

```
class Mahasiswa {  
    String nama;  
    String nim;  
    String prodi;  
}
```

Pada contoh di atas, class Mahasiswa memiliki tiga atribut, yaitu nama, nim, dan prodi. Class ini belum memiliki method, tetapi sudah cukup untuk menunjukkan bahwa sebuah class dapat digunakan untuk menyimpan data yang berkaitan dengan suatu objek.

## 2.6 Object

Object adalah hasil instansiasi dari class. Jika class adalah cetak biru, maka object adalah hasil nyata yang dibuat berdasarkan cetak biru tersebut.

Sebagai contoh, dari class Mahasiswa, kita bisa membuat banyak object. Setiap object akan memiliki nilai atribut yang berbeda, meskipun semuanya berasal dari class yang sama.

Contoh program:

```
class Mahasiswa {  
  
    String nama;  
  
    String nim;  
  
    String prodi;  
  
}
```

```
public class MainMahasiswa {  
  
    public static void main(String[] args) {  
  
        Mahasiswa mhs1 = new Mahasiswa();  
  
        mhs1.nama = "Andi Saputra";  
        mhs1.nim = "123140001";  
        mhs1.prodi = "Informatika";  
  
        System.out.println("Nama   : " + mhs1.nama);  
        System.out.println("NIM    : " + mhs1.nim);  
        System.out.println("Prodi  : " + mhs1.prodi);  
  
    }  
  
}
```

Output:

**Nama : Andi Saputra**

**NIM : 123140001**

**Prodi : Informatika**

Contoh di atas menunjukkan bahwa object `mhs1` dibuat dari class `Mahasiswa`. Setelah object dibuat, atribut-atributnya dapat diisi, lalu nilainya dapat ditampilkan ke layar.

## 2.7 Atribut dan Method

Dalam OOP, objek biasanya memiliki dua hal utama, yaitu **atribut** dan **method**. Atribut menyimpan data, sedangkan method mendefinisikan perilaku atau aksi yang bisa dilakukan objek.

Pada class Mahasiswa, atribut dapat berupa nama, nim, dan prodi. Sementara itu, method dapat berupa `tampilkanData()`, yang bertugas menampilkan isi atribut ke layar.

Contoh program:

```
class Mahasiswa {  
  
    String nama;  
  
    String nim;  
  
    String prodi;  
  
    void tampilkanData() {  
  
        System.out.println("Nama   : " + nama);  
  
        System.out.println("NIM    : " + nim);  
  
        System.out.println("Prodi  : " + prodi);  
  
    }  
  
}
```

```
public class DemoMahasiswa {  
  
    public static void main(String[] args) {  
  
        Mahasiswa mhs1 = new Mahasiswa();  
  
        mhs1.nama = "Citra Lestari";  
  
        mhs1.nim = "123140002";  
  
        mhs1.prodi = "Informatika";  
  
        mhs1.tampilkanData();  
  
    }  
  
}
```

Output:

```
Nama   : Citra Lestari
NIM    : 123140002
Prodi  : Informatika
```

Dengan menambahkan method, class menjadi lebih terstruktur. Kita tidak perlu menulis `System.out.println()` berulang-ulang di `main`, karena tugas tersebut sudah dibungkus di dalam method `tampilkanData()`.

## 2.8 Field

Field adalah variabel yang menjadi anggota dari sebuah class. Dengan kata lain, field adalah atribut yang disimpan di dalam class untuk merepresentasikan data dari objek.

Pada praktik dasar, mahasiswa cukup memahami bahwa field adalah data milik class. Field dapat berupa data yang nilainya berbeda untuk setiap object, atau data yang nilainya sama untuk semua object jika menggunakan `static`.

Contoh field biasa:

```
class Mahasiswa {
    String nama;
    String nim;
}
```

Pada contoh di atas, `nama` dan `nim` adalah field. Jika dibuat dua object berbeda, maka masing-masing object akan memiliki nilai `nama` dan `nim` sendiri.

Contoh program:

```
class Laboratorium {

    String namaLab;

    int kapasitas;

}
```

```
public class DemoField {  
  
    public static void main(String[] args) {  
  
        Laboratorium lab1 = new Laboratorium();  
  
        lab1.namaLab = "Lab Pemrograman";  
  
        lab1.kapasitas = 40;  
  
  
        System.out.println("Nama Lab   : " + lab1.namaLab);  
  
        System.out.println("Kapasitas : " + lab1.kapasitas);  
  
    }  
  
}
```

## 2.9 Method

Method adalah sekumpulan instruksi yang diletakkan di dalam class untuk melakukan tugas tertentu. Dengan method, program menjadi lebih rapi karena suatu pekerjaan dapat dibungkus dalam satu bagian yang dapat dipanggil berulang kali.

Dalam Java, method bisa dibuat tanpa parameter, dengan parameter, dan bisa juga mengembalikan nilai. Pada tahap awal, mahasiswa perlu memahami ketiganya secara bertahap.

### 2.9.1 Method tanpa Parameter

Method tanpa parameter tidak membutuhkan nilai masukan saat dipanggil. Method seperti ini cocok untuk menampilkan informasi atau menjalankan tugas yang sudah tetap.



Contoh:

```
public class ContohMethod1 {  
  
    static void sapa() {  
        System.out.println("Halo mahasiswa Informatika ITERA");  
        System.out.println("Selamat belajar PBO");  
    }  
  
    public static void main(String[] args) {  
        sapa();  
    }  
}
```

### 2.9.2 Method dengan Parameter

Method dengan parameter menerima masukan saat dipanggil. Dengan cara ini, method menjadi lebih fleksibel karena dapat digunakan untuk banyak data yang berbeda.

Contoh:

```
public class ContohMethod2 {  
  
    static void tampilNama(String nama) {  
        System.out.println("Halo, " + nama);  
    }  
  
    public static void main(String[] args) {  
        tampilNama("Rafi");  
        tampilNama("Dewi");  
    }  
}
```

Output :

```
Halo, Rafi  
Halo, Dewi
```

### 2.9.3 Method dengan Nilai Kembalian

Method juga bisa mengembalikan nilai menggunakan `return`. Method seperti ini biasanya digunakan untuk perhitungan atau pengolahan data.

Contoh:

```
public class ContohMethod3 {  
  
    static int hitungJumlah(int a, int b) {  
        return a + b;  
    }  
  
    public static void main(String[] args) {  
        int hasil = hitungJumlah(10, 15);  
        System.out.println("Hasil penjumlahan = " + hasil);  
    }  
}
```

Method sangat penting dalam OOP karena hampir semua perilaku objek didefinisikan melalui method. Semakin baik mahasiswa memahami method, semakin mudah juga memahami class dan object.

## 2.10 Java Modifier

Modifier adalah kata kunci yang digunakan untuk mengatur hak akses atau sifat dari class, atribut, dan method. Pada tahap awal, modifier yang paling penting untuk dipahami adalah `public`, `private`, dan `protected`, ditambah `static` dan `final` secara sederhana.

### 2.10.1 Public

Jika suatu class, atribut, atau method diberi modifier `public`, maka ia dapat diakses dari bagian program lain yang sesuai. Modifier ini biasanya digunakan pada method yang memang ingin dipakai dari luar class.

### 2.10.2 Private

Jika suatu atribut diberi modifier `private`, maka atribut tersebut tidak bisa diakses langsung dari luar class. Ini berguna untuk menjaga data agar lebih aman dan lebih terkontrol.

### 2.10.3 Protected

Modifier `protected` memberi akses yang lebih terbatas dibanding `public`, tetapi lebih longgar dibanding `private`. Pada tahap awal, mahasiswa cukup mengenal dulu bahwa `protected` sering dipakai ketika nanti masuk ke inheritance.

### 2.10.4 Static

Modifier `static` berarti anggota tersebut milik class, bukan milik object tertentu. Pada program awal, mahasiswa paling sering melihat `static` pada `main`.

### 2.10.5 Final

Modifier `final` berarti nilainya tidak bisa diubah lagi. Ini sering dipakai untuk konstanta.

Contoh program sederhana:

```
class Kampus {
    public String namaKampus = "ITERA";
    private String lokasi = "Lampung Selatan";

    public void tampilkanInfo() {
        System.out.println("Nama Kampus : " + namaKampus);
        System.out.println("Lokasi      : " + lokasi);
    }
}
```

```
public class DemoModifier {
    public static void main(String[] args) {
        Kampus k = new Kampus();
        k.tampilkanInfo();
    }
}
```

Pada contoh di atas, `lokasi` dibuat `private`, sehingga lebih aman disimpan di dalam class. Data tersebut tetap bisa ditampilkan melalui method `tampilkanInfo()`.

## 2.11 Keyword this

Keyword `this` digunakan untuk merujuk pada object yang sedang aktif. Biasanya `this` dipakai ketika nama parameter sama dengan nama atribut, sehingga Java dapat membedakan mana atribut milik object dan mana parameter dari method atau constructor.

Contoh program:

```
class Mahasiswa {  
    String nama;  
    String nim;  
  
    void isiData(String nama, String nim) {  
        this.nama = nama;  
        this.nim = nim;  
    }  
  
    void tampilkanData() {  
        System.out.println("Nama : " + nama);  
        System.out.println("NIM : " + nim);  
    }  
}
```

```
public class DemoThis {  
    public static void main(String[] args) {  
        Mahasiswa m1 = new Mahasiswa();  
        m1.isiData("Budi Pratama", "123140003");  
        m1.tampilkanData();  
    }  
}
```

Output:

```
Nama : Budi Pratama  
NIM : 123140003
```

Jika `this` tidak digunakan pada kondisi seperti ini, Java akan lebih sulit membedakan atribut dan parameter yang memiliki nama sama. Karena itu, `this` sangat membantu agar kode tetap jelas dan mudah dibaca.

## 2.12 Constructor

Constructor adalah method khusus yang digunakan untuk menginisialisasi object saat object dibuat. Nama constructor harus sama dengan nama class, dan constructor tidak memiliki tipe data kembalian, bahkan `void` sekalipun.

Constructor sangat berguna karena object bisa langsung memiliki nilai awal saat dibuat. Dengan begitu, programmer tidak perlu selalu mengisi atribut satu per satu setelah object diciptakan.

Contoh constructor sederhana:

```
class Mahasiswa {
    String nama;
    String nim;
    String prodi;

    Mahasiswa(String nama, String nim, String prodi) {
        this.nama = nama;
        this.nim = nim;
        this.prodi = prodi;
    }

    void tampilkanData() {
        System.out.println("Nama : " + nama);
        System.out.println("NIM : " + nim);
        System.out.println("Prodi : " + prodi);
    }
}
```

```
public class DemoConstructor {
    public static void main(String[] args) {
        Mahasiswa m1 = new Mahasiswa("Salsa Putri", "123140004",
        "Informatika");
        m1.tampilkanData();
    }
}
```

Output:

```
Nama : Salsa Putri
NIM : 123140004
Prodi : Informatika
```

### 2.12.1 Constructor Default

Jika programmer tidak membuat constructor sendiri, Java dapat menyediakan constructor default. Constructor ini tidak memiliki parameter dan digunakan untuk membuat object dengan nilai atribut yang masih default.

Contoh :

```
class Buku {
    String judul;
    String penulis;
}
```

```
public class DemoConstructorDefault {
    public static void main(String[] args) {
        Buku b1 = new Buku();

        b1.judul = "Pemrograman Java Dasar";
        b1.penulis = "Tim Praktikum";

        System.out.println("Judul    : " + b1.judul);
        System.out.println("Penulis  : " + b1.penulis);
    }
}
```

### 2.12.2 Constructor Overloading

Satu class bisa memiliki lebih dari satu constructor, asalkan parameternya berbeda. Hal ini disebut constructor overloading.

Contoh:

```
class MataKuliah {
    String namaMk;
    int sks;

    MataKuliah() {
        namaMk = "Pemrograman Berorientasi Objek";
        sks = 3;
    }

    MataKuliah(String namaMk, int sks) {
        this.namaMk = namaMk;
        this.sks = sks;
    }

    void tampilkanData() {
        System.out.println("Mata Kuliah : " + namaMk);
        System.out.println("SKS          : " + sks);
    }
}
```

```

public class DemoOverloadingConstructor {
    public static void main(String[] args) {
        MataKuliah mk1 = new MataKuliah();
        MataKuliah mk2 = new MataKuliah("Struktur Data", 3);

        mk1.tampilkanData();
        System.out.println();
        mk2.tampilkanData();
    }
}

```

## 2.13 Contoh Program Lengkap

Berikut contoh program yang menggabungkan class, object, method, constructor, dan this dalam satu program sederhana bertema data mahasiswa ITERA.

```

class Mahasiswa {
    String nama;
    String nim;
    String prodi;
    int semester;

    Mahasiswa(String nama, String nim, String prodi, int semester) {
        this.nama = nama;
        this.nim = nim;
        this.prodi = prodi;
        this.semester = semester;
    }

    void tampilkanData() {
        System.out.println("Nama      : " + nama);
        System.out.println("NIM      : " + nim);
        System.out.println("Prodi    : " + prodi);
        System.out.println("Semester : " + semester);
    }

    void sapa() {
        System.out.println("Halo, saya " + nama + " dari Prodi " + prodi
+ ".");
    }
}

```

```
public class DemoMahasiswaITERA {  
    public static void main(String[] args) {  
        Mahasiswa m1 = new Mahasiswa("Andi Saputra", "123140001",  
"Informatika", 2);  
        Mahasiswa m2 = new Mahasiswa("Dewi Lestari", "123140002",  
"Informatika", 2);  
  
        m1.tampilkanData();  
        m1.sapa();  
  
        System.out.println();  
  
        m2.tampilkanData();  
        m2.sapa();  
    }  
}
```

Contoh ini sangat baik untuk latihan praktikum karena mahasiswa dapat langsung melihat bentuk nyata OOP. Dalam satu program saja, mereka bisa melihat bagaimana object dibuat, bagaimana constructor mengisi data, dan bagaimana method dipanggil untuk menampilkan perilaku objek.



## **2.14 Tugas Percobaan**

### **2.14.1 Percobaan 1**

Buatlah class Mahasiswa yang memiliki atribut:

- nama
- nim
- prodi

Lalu buat satu object dari class tersebut dan tampilkan seluruh datanya ke layar.

### **2.14.2 Percobaan 2**

Modifikasi class Mahasiswa dengan menambahkan method tampilkanData().  
Gunakan method tersebut untuk menampilkan data object yang telah dibuat.

### **2.14.3 Percobaan 3**

Buatlah class Dosen yang memiliki atribut:

- nama
- nidn
- prodi

Tambahkan method tampilkanData() lalu buat dua object dosen yang berbeda.

### **2.14.4 Percobaan 4**

Buatlah class MataKuliah yang memiliki:

- atribut namaMk
- atribut sks
- constructor untuk mengisi data
- method tampilkanData()

Lalu buat dua object mata kuliah dengan data yang berbeda dan tampilkan hasilnya.

### 2.14.5 Percobaan 5

Buatlah program sederhana bertema **Data Mahasiswa ITERA** yang memiliki ketentuan berikut:

1. Gunakan class `Mahasiswa`
2. Gunakan constructor untuk mengisi data
3. Gunakan method `tampilkanData()`
4. Buat minimal 3 object mahasiswa
5. Tampilkan seluruh data mahasiswa ke layar

### 2.15 Pertanyaan Diskusi

1. Apa yang dimaksud dengan pemrograman berorientasi objek?
2. Apa perbedaan antara class dan object?
3. Apa fungsi atribut dan method dalam sebuah class?
4. Mengapa constructor memudahkan pembuatan object?
5. Apa kegunaan keyword `this` dalam Java?
6. Apa perbedaan antara field dan variabel lokal?
7. Mengapa modifier `private` sering digunakan untuk atribut?
8. Mengapa pendekatan OOP dianggap lebih terstruktur dibanding pendekatan prosedural?

## **2.16 Tugas Laporan Praktikum**

Mahasiswa diminta membuat laporan praktikum dengan format berikut. Format ini dibuat konsisten dengan pola pelaporan pada BAB 1 agar alur praktikum tetap seragam.

### **2.16.1 1. Cover**

Berisi:

- Judul praktikum
- Nama
- NIM
- Program Studi
- Mata Kuliah
- Dosen Pengampu
- Kelas

### **2.16.2 2. Tujuan Praktikum**

Tuliskan tujuan praktikum BAB 2 sesuai dengan materi yang dikerjakan.

### **2.16.3 3. Alat dan Bahan**

Tuliskan perangkat yang digunakan, misalnya:

- Laptop atau PC
- JDK
- NetBeans
- Modul praktikum

### **2.16.4 4. Langkah Percobaan**

Lakukan percobaan pada subbab **2.14 Tugas Percobaan**. Tuliskan langkah kerja setiap percobaan secara runtut, mulai dari membuat class, menulis kode, menjalankan program, hingga melihat output.

### **2.16.5 5. Source Code**

Lampirkan setiap source code yang dibuat pada masing-masing percobaan. Source code harus rapi dan diberi nama file yang sesuai.

### **2.16.6 6. Hasil Output Program**

Tampilkan hasil output dari setiap percobaan. Jika perlu, sertakan tangkapan layar hasil running program.

### **2.16.7 7. Analisis Program**

Berikan penjelasan untuk setiap percobaan. Analisis minimal memuat:

- class yang dibuat
- atribut yang digunakan
- method yang digunakan
- object yang diinstansiasi
- hasil yang diperoleh dari program

### **2.16.8 8. Kesimpulan**

Tuliskan kesimpulan singkat mengenai apa yang dipelajari pada praktikum BAB 2. Kesimpulan sebaiknya menegaskan pemahaman mahasiswa tentang class, object, method, constructor, dan `this`.

Laporan dikumpulkan dalam bentuk **PDF** melalui GCR yang disediakan asisten praktikum.

## 2.17 Penutup

Pada BAB 2 ini, mahasiswa mulai masuk ke inti dari pemrograman berorientasi objek. Materi seperti `class`, `object`, `atribut`, `method`, `field`, `modifier`, `this`, dan `constructor` merupakan pondasi yang sangat penting untuk bab-bab berikutnya.

Jika pondasi ini benar-benar dipahami, maka materi seperti `encapsulation`, `inheritance`, dan `polymorphism` pada pertemuan selanjutnya akan terasa jauh lebih mudah. Karena itu, mahasiswa sebaiknya tidak hanya membaca contoh, tetapi juga benar-benar mengetik ulang, menjalankan program, dan mencoba memodifikasi sendiri setiap percobaan yang ada.